

Relatório Individual — A2 Parte 2

Anthony Guilherme Cazuni da Silva

Qual foi sua principal contribuição no trabalho?

Cuidei da base do projeto: a configuração do banco de dados, a camada de Repository, os middlewares de autenticação e permissão, e a configuração de sessão no server.js. Foi a fundação sobre a qual a Alice e o Gabriel construíram as outras camadas.

Quais partes do projeto você desenvolveu?

Desenvolvi o config/database.js, os quatro repositories incluindo as consultas com JOIN para a listagem de matrículas, o authMiddleware e o permissaoMiddleware, além de toda a configuração do Express (sessão, method-override, arquivos estáticos e tratamento de 404).

Qual foi o trecho mais complicado do código? Explique o motivo.

A persistência do banco. Usamos sql.js para evitar dependências nativas de compilação, mas ele roda em memória se eu não salvasse em disco, tudo se perdia ao reiniciar. Tive que criar uma função persist() que exporta o banco para um arquivo .db e chamar depois de cada INSERT/UPDATE/DELETE.

Qual foi sua maior dificuldade técnica e como resolveu?

Fazer o permissaoMiddleware funcionar como uma factory. Eu precisava de um middleware que recebesse o perfil exigido (ADMIN) como parâmetro, mas middleware do Express tem assinatura fixa (req, res, next). A solução foi criar uma função que retorna outro middleware: permissao('ADMIN') devolve a função que o Express realmente executa. Entender closures foi o que destravou isso.

Você utilizou IA em alguma parte? Se sim, explique como.

Usei o GitHub Copilot enquanto escrevia os repositories ele complementava os comandos SQL depois que eu escrevia o primeiro como modelo, o que economizou tempo nos CRUDs parecidos, aí então usei o ChatGPT para entender a API do sql.js porque a documentação dele é diferente do SQLite tradicional.

O que você aprendeu de novo com este trabalho?

Apreendi o conceito de middleware e como o Express encadeia as funções com next(), algo que eu só tinha visto de forma superficial. Também entendi melhor porque isolar todo o SQL em uma única camada vale a pena: se quiséssemos trocar de banco, só o repository mudaria. E aprendi a importância de seeds para que o time todo rode o projeto com os mesmos dados de teste.

Se tivesse mais tempo, o que melhoraria no projeto?

Migraria a persistência para um SQLite nativo ou PostgreSQL, eliminando o persist() manual e ganhando transações de verdade. Também adicionaria índices nas colunas mais consultadas (CPF e aluno_id) e um sistema de migrações para versionar o schema, em vez de criar as tabelas direto no código.